WEEK 04

xv6 Setup & Codebase Exploration

Theory Pack

What is an operating system?
User space vs kernel space.
xv6 source tree and key data structures.

Then write your first xv6 program.

INSIDE

SECTION 01

What is an Operating System?

An operating system is a layer of software that sits between hardware and user programs. It has two main jobs: resource management and abstraction.

Resource management: the CPU, memory, disk, and devices are shared among running programs. The OS decides who gets what, when, and for how long.

Abstraction: the OS hides the messy details of hardware behind clean interfaces. A file is easier to work with than a spinning disk sector. A process is easier to work with than a CPU core and register file.

Three core abstractions

Abstraction	What it hides	What the OS provides
Process	CPU, registers, interrupts	The illusion that each program owns the CPU
Address space	RAM, page tables	Each program gets its own private memory
File	Disk sectors, block caches	Named, persistent byte sequences

Every OS you have ever used — Linux, Windows, macOS — provides these three. xv6 provides them too, in about 10,000 lines of C.

SECTION 02

User Space vs Kernel Space

Processors provide different privilege levels. xv6 runs on RISC-V, which has three: U-mode (user), S-mode (supervisor/kernel), and M-mode (machine). Your programs run in U-mode; the kernel runs in S-mode.

Privilege levels

Mode	Name	What you can do
U-mode	User mode	Run your code, access your own memory
S-mode	Supervisor mode	Configure page tables, handle interrupts, control devices
M-mode	Machine mode	Change the CPU's physical configuration (rarely used)

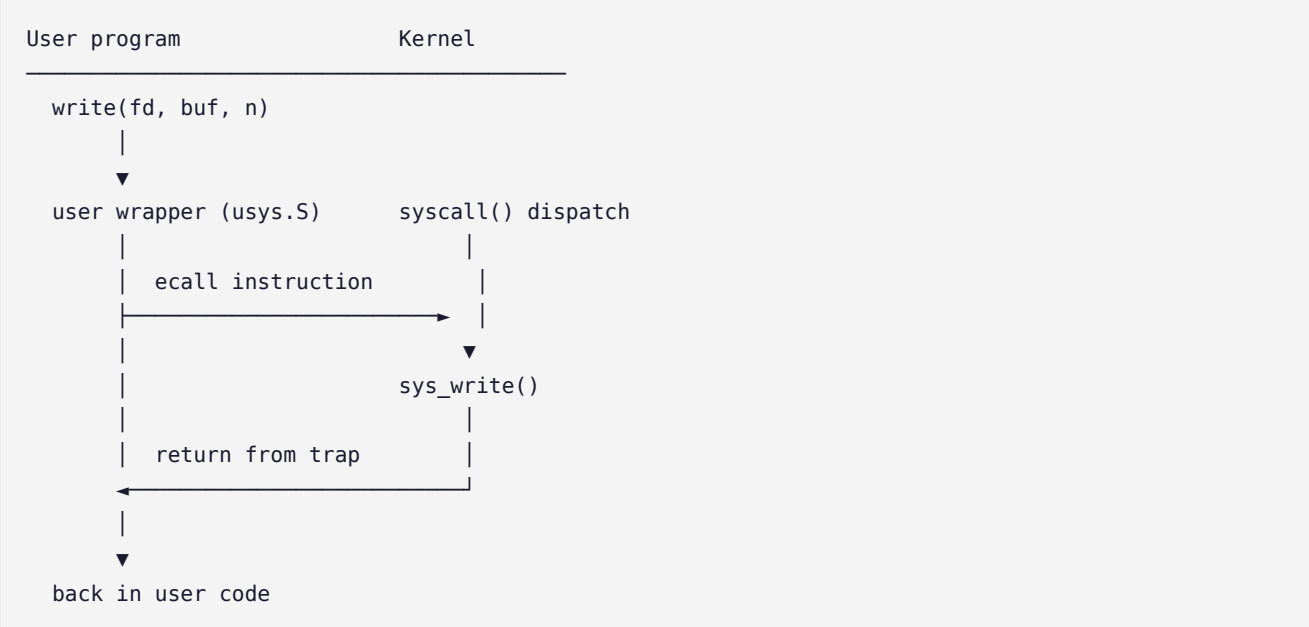
Why the separation?

Without it, a bug in any program could crash the entire machine. A malicious program could read another program's files, steal passwords, or wipe the disk. The hardware-enforced boundary limits the damage.

Crossing the boundary: system calls

When a user program needs a privileged operation — reading a file, creating a process — it cannot do it directly. It asks the kernel via a system call.

The mechanism on RISC-V: the ecall instruction raises privilege from U-mode to S-mode, saves the program counter, sets scause to 'environment call from U-mode', and jumps to the trap handler address in stvec. The kernel handles the request, then returns via sret, which restores U-mode and resumes the user program exactly where it left off.



> *A system call is a controlled crossing from user space into kernel space, and the hardware makes sure you can only cross at designated entry points.*

SECTION 03

Introduction to xv6

xv6 is a reimplementation of Sixth Edition Unix (V6) for modern RISC-V processors, developed at MIT. It is small (~10,000 lines total), complete (processes, virtual memory, file system, pipes, device drivers), modifiable, and runnable on QEMU or real RISC-V hardware.

The source tree

```
xv6-riscv/
├── kernel/          # OS kernel code
│   ├── proc.h       # Process table and PCB (struct proc)
│   ├── proc.c       # fork, scheduler, wait, exit
│   ├── syscall.h    # System call numbers
│   ├── syscall.c    # Dispatch table
│   ├── trap.c       # Trap handling: usertrap, kerneltrap
│   ├── vm.c         # Virtual memory: page tables
│   ├── kalloc.c     # Physical memory allocator
│   ├── fs.c         # File system implementation
│   ├── pipe.c       # Pipe implementation
│   └── main.c       # Kernel initialization sequence
│
├── user/            # User-space programs
│   ├── user.h       # System call declarations
│   ├── sh.c         # The xv6 shell
│   └── ...          # ls, cat, echo, forktest, ...
│
└── Makefile         # Build system — find UPROGS here
```

Key data structures

struct proc — the Process Control Block (kernel/proc.h). This is the kernel's record for each running process.

```
struct proc {
    struct spinlock lock;
    enum procstate state;   // UNUSED, SLEEPING, RUNNABLE, RUNNING, ZOMBIE
    void *chan;             // sleep channel (if SLEEPING)
    int pid;
    struct proc *parent;
    pagetable_t pagetable;  // user page table
    uint64 kstack;          // kernel stack address
    uint64 sz;              // process memory size
    struct trapframe *trapframe;
    struct context context; // for context switching
    struct file *ofile[NOFILE];
    struct inode *cwd;      // current working directory
    char name[16];
};
```

Process states (kernel/proc.h): UNUSED — slot is free; SLEEPING — waiting for an event (e.g. disk I/O); RUNNABLE — ready to run, waiting for CPU; RUNNING — currently executing on a CPU; ZOMBIE — exited but not yet waited on by parent.

SECTION 04

The Build Process

When you run make, several steps happen in sequence: cross-compilation, user program compilation, file system image creation, and QEMU launch.

Steps

- Cross-compilation: gcc-riscv64-unknown-elf compiles every .c file in kernel/ to RISC-V object files, then links them into kernel/kernel (the OS binary).
- User programs: every program listed in UPROGS in the Makefile is compiled into a RISC-V executable.
- File system image: the mkfs tool packages all user programs into fs.img along with a README and any other static files. This is the 'disk' that xv6 boots with.
- QEMU launch: QEMU loads xv6.img (the kernel binary) and fs.img (the root file system).

> *Important: user programs are baked into fs.img at build time. You cannot add a program at runtime — it must be in UPROGS before you run make.*

SECTION 05

Writing Your First xv6 Program

Every xv6 user program follows a simple template. It includes only headers from the xv6 tree, calls exit(0), and links against the xv6 user library.

The program template

```
#include "kernel/types.h"    // basic type definitions
#include "user/user.h"       // system call declarations

int main(int argc, char *argv[]) {
    // your code here
    exit(0);                 // every program must call exit!
}
```

Key differences from Linux user-space programming

- No #include <stdio.h> — use kernel-provided headers.
- printf comes from xv6's own library (declared in user/user.h).
- exit(0) is mandatory — falling off the end of main is not defined.
- Standard library is minimal: no malloc, no fopen, just system calls.

Adding your program to the build

```
# 1. Save file as user/myprog.c
# 2. In Makefile, add to UPROGS:
        $U/_myprog\
# 3. Rebuild:
        make clean && make && make qemu
# 4. Run in QEMU:
        $ myprog
```

SECTION 06

References

The following resources are recommended for Week 4. The xv6 book chapters 1 and 2 cover exactly the material you need.

- xv6-riscv book — Chapters 1 (OS overview), 2 (xv6 architecture): https://pdos.csail.mit.edu/6.828/2025/xv6/book-riscv-rev4.pdf
- xv6-riscv source — the repo you cloned: https://github.com/mit-pdos/xv6-riscv
- OSTEP — Chapters 2, 4, 5 for OS concepts: https://pages.cs.wisc.edu/~remzi/OSTEP/
- RISC-V Privileged Architecture — ecall, sret, stvec documentation.

> *Stuck? Re-read this theory.pdf. If QEMU hangs, press Ctrl-A then X to quit QEMU. If your program panics the kernel, add printf calls before the crash to narrow it down. If you still get stuck, message your mentor.*